

GDC Resolve Encoder

Complete guide: installation, settings explained, examples,
troubleshooting

by Cristi Gordas

Guide version: v1.0 — matches plugin v1.0.0

1. What is GDC Resolve Encoder

GDC Resolve Encoder is a native plugin for DaVinci Resolve (Studio and Free) that adds extra export codecs, powered by FFmpeg — H.264 and H.265, in software (x264/x265) and hardware (GPU acceleration, where available) variants, on Mac, Windows, and Linux.

It doesn't change Resolve's interface or add editing features. It does one thing: it shows up as an extra Codec option on the Deliver page, alongside Resolve's own native H.264/H.265, but with much finer control over quality, speed, and final file size.

What it can do

- H.264 and H.265 export, software (CPU) and hardware (if your machine has compatible acceleration).
- Fine quality control: CRF (constant quality), Target Bitrate, or QP (constant quantizer).
- Speed presets (from ultrafast to veryslow) — the speed/compression-efficiency trade-off.
- H.264 profiles (baseline/main/high/high422) for compatibility with older players/devices.
- Tune — content-specific optimizations (animation, film grain, etc).
- 8-bit and 10-bit export (High10/Main10) for anyone who needs extra color depth.

What it can't do (yet)

- No HDR support (PQ/HLG metadata) — planned for a future version.
- 10-bit is available only on the software variants (x264/x265), not hardware — hardware 10-bit support varies too much across GPUs to enable it universally right now.
- No multi-pass rendering — currently single-pass only.
- Doesn't replace Resolve's native codecs — it adds to them, as an extra option.

Note: If you need a feature that's missing from this list, let the developer know — the list above reflects the current version, not a permanent ceiling.

2. Installation

macOS

Download the Mac archive from the Releases page. Unzip it — you'll find the plugin bundle, an install script (install.sh), and a quick-start text file.

Simplest way: open Terminal in that folder and run:

```
./install.sh
```

The script automatically checks whether FFmpeg is installed (installs it via Homebrew if missing), removes the macOS quarantine flag (needed because the binary isn't Apple-notarized), and copies the plugin to the correct system location. It will ask for your password once.

Manual install, if you prefer (macOS)

```
xattr -rd com.apple.quarantine gdc_resolve_encoder.dvcp.bundle  
cp -R gdc_resolve_encoder.dvcp.bundle "/Library/Application Support/Blackmagic Design/DaVinci Resolve/IOPlugins/"
```

Windows

Download the Windows archive, unzip it. Copy the bundle folder (ending in .dvcp.bundle) to:

```
%ProgramData%\Blackmagic Design\DaVinci Resolve\Support\IOPlugins\
```

The required FFmpeg DLLs are already included in the archive — you don't need to install FFmpeg separately on Windows.

Linux

Download the Linux archive, unzip it. Copy the bundle into your Resolve installation's IOPlugins directory (typically under /opt/resolve/IOPlugins/ or similar, depending on your distribution). FFmpeg needs to already be installed on the system (most distributions have it, or it's a quick package-manager install).

After installing, on any platform

- Fully restart DaVinci Resolve (quit and reopen the whole application).
- Deliver page → Format: QuickTime or MP4 → Codec: look for the new option in the list.
- If it doesn't show up, see the Troubleshooting section below.

3. Every setting explained

Codec / Type — which one should I pick?

Variant	When to use it
H.264 Software	Most compatible, works everywhere. Safe default choice.
H.264 Hardware	Much faster export, slightly lower quality at the same bitrate. Good for quick previews.
H.265 Software	Files roughly 30-40% smaller than H.264 at the same quality. Slower export.
H.265 Hardware	Fast and small files — best trade-off if your machine supports it.
10-bit (High10/Main10)	Only if your source has real 10-bit depth and you want to preserve it (grading, HDR prep).

Preset — speed vs efficiency

The preset does NOT directly change visible quality — it changes how much time the encoder spends searching for the best compression at the requested quality. A slower preset ("slow", "veryslow") produces smaller files at the same visual quality, but takes much longer to export.

Preset	Recommended for
ultrafast / superfast	Quick tests, previews — not for final delivery.
fast / medium	Good balance — medium is the default choice, suitable for most cases.
slow / slower	Final delivery when export time isn't a concern and you want a smaller file.
veryslow	Maximum compression — only if you have time to spare.

Rate Control — CRF, Bitrate, or QP?

These are three different ways of telling the encoder "how much to compress". They don't combine — pick just one.

CRF (Constant Quality) — recommended for most cases

You tell the encoder "I want this quality level", and it decides how much bitrate to use, frame by frame — more on complex scenes, less on simple ones. Result: consistent visual quality, variable and unpredictable file size ahead of time.

CRF value	Result
0	Lossless — huge files, only for archival/master purposes.
18-20	Visually identical to source, to the human eye.
23	The default value — good balance for most deliveries.
28-30	Noticeably more compressed — suitable for previews or quick web use.
51	Lowest possible quality.

Bitrate (Target Bitrate)

You tell the encoder exactly how many kbps to use, regardless of content. Useful when you have a strict file-size limit (e.g. a platform that only accepts up to X MB) or a fixed-bitrate delivery requirement (broadcast, streaming).

Note: With a fixed Bitrate, a very complex scene (lots of motion, detail) can look worse than the same scene exported with CRF, because the encoder isn't allowed to "spend" more on it.

QP (Constant Quantizer)

Similar to CRF on the same scale (0-51, lower = better), but fully disables adaptive adjustment — every frame gets the exact same compression level regardless of content. Rarely used in normal production; mostly useful for technical testing/comparisons or very specific workflow requirements.

Profile (H.264 only, 8-bit, software)

Profile	When to use it
baseline	Maximum compatibility with very old/simple devices. Rarely needed today.
main	Broad compatibility, without advanced compression features.
high	Default choice — best compression, compatible with almost any modern player.
high422	Only if you explicitly need 4:2:2 subsampling (specific broadcast workflows).

Tune — content-type optimizations

Optional. Fine-tunes the encoder for a specific type of material. If unsure, leave it on "none" — CRF/preset already do a good job for normal content.

Tune	For
film	Normal filmed content with natural film grain (H.264 only).
animation	Animation/cartoons — better preserves sharp lines and flat color areas.
grain	Source with visible, pronounced grain — tries to preserve it more faithfully.
stillimage	Exporting a single still image, not video (H.264 only).
psnr / ssim	Optimizes for technical quality metrics — mostly used in testing, not production.
fastdecode	Prioritizes fast playback decoding, on weaker hardware.
zerolatency	Live streaming, where delay needs to be minimized.

4. Practical recipes — recommended combinations

If you'd rather not analyze every setting individually, here are ready-made combinations for common situations.

Client delivery, maximum quality, reasonable file size

- Codec: H.264 Software (or H.265 Software if the client accepts smaller files)
- Preset: slow
- Rate Control: CRF
- CRF: 18-20
- Profile: high
- Tune: none (or "film" if the source has visible grain)

Quick preview for client approval

- Codec: H.264 Hardware (if available) — much faster export
- Preset: fast (if using the software variant)
- Rate Control: CRF
- CRF: 26-28

Export for YouTube/web

- Codec: H.264 Software
- Preset: medium or slow
- Rate Control: CRF
- CRF: 20-23
- Profile: high

Archival / master, disk space isn't a concern

- Codec: H.265 Software 10-bit (if the source has real 10-bit depth)
- Preset: slow or veryslow
- Rate Control: CRF
- CRF: 16-18

Smallest possible file, quality is secondary

- Codec: H.265 Software (considerably smaller files than H.264)
- Preset: medium
- Rate Control: Bitrate
- Bitrate: set explicitly to the limit you need

5. Common issues and solutions

The codec doesn't show up in the Deliver list at all

- Did you fully restart DaVinci Resolve after installing? (not just closing the project — the whole application)
- On Mac: did you run `install.sh` or manually remove the quarantine with `xattr`? Without that, macOS silently blocks the plugin.
- Check the exact folder: the bundle needs to be directly inside `IOPlugins`, not in an extra subfolder.
- The selected Format needs to be QuickTime or MP4 — the codec doesn't appear for other formats.

Rendering fails immediately, without a visible explanation

- Check you have enough free disk space at the destination.
- Try a standard resolution (1920x1080) as a test — if that works, the issue may be related to an unusual project resolution.
- If you have other encoder plugins installed, try temporarily disabling the rest to rule out conflicts.

The resulting file has audio only, no video

- Update to the latest plugin version — this was a known issue that has since been fixed.
- Check that you've explicitly selected a correct Video codec (not just Audio) in the Export Video section of Deliver.

Export is very slow

- Is the preset set to "slow" or "veryslow"? Try "medium" or "fast".
- The hardware variant (if available on your machine) is significantly faster than software.
- H.265 is inherently slower to encode than H.264 at the same processing power.

Colors look slightly different from what you see in Resolve

- Check whether the color range (video/full) matches what the player you're checking the file with expects — some players misinterpret the range if it's not detected correctly.
- If exporting 10-bit, make sure your player/monitor actually supports 10-bit — otherwise you're seeing the system's automatic conversion, not the original.

6. Licenses and attribution

This plugin's own code is licensed under the MIT License.

The plugin uses FFmpeg (libavcodec, libavutil, libswscale), licensed GPL v2+ in the configuration used here, as well as libx264 and libx265, both GPL-licensed. Because of linking against GPL libraries, this plugin's compiled binary is, as a distribution, subject to GPL terms — regardless of the license of the plugin's own code.

Note: This document is not legal advice. For full details and legal implications — especially if you intend large-scale commercial distribution — see the `THIRD_PARTY_LICENSES` document in the repository, or a lawyer specialized in open-source licensing.

- FFmpeg — <https://ffmpeg.org> — © FFmpeg contributors
- libx264 — <https://www.videolan.org/developers/x264.html> — © x264 project contributors
- libx265 — <https://www.videolan.org/developers/x265.html> — © MulticoreWare Inc. and contributors